Software Requirements Specification (SRS)

Project: QuickBite — Cloud-Native Distributed Food Delivery & Intelligent Dispatch Platform

> **Course Case Study**: Feng Chia University — Software Engineering (ASE) Practical Demonstration Material
> **Version**: v2.4 (Official Release)
> **Aligned Slide Modules**: Ch03 Requirements Engineering & Ch04 Object-Oriented System Modeling
> **Standard Compliance**: Conforms to IEEE 830 / ISO/IEC/IEEE 29148 Software Requirements Specification Standard

---

1. Introduction & Project Scope

1.1 Project Background & Vision

"QuickBite" is a modern, high-throughput cloud-native food delivery and real-time dispatch e-commerce platform. It seamlessly connects three core actors: **Customers**, **Merchant Restaurants**, and **Courier Partners**. Through real-time bipartite matching algorithms, geospatial indexing, and event-driven architecture, QuickBite guarantees a reliable "30-minute hot meal delivery" experience.

As the central case study for the Software Engineering course, this project demonstrates:

1. **The Requirements Triad**: Harmonizing Domain Properties, Software Specifications, and User Requirements.
2. **Multi-Perspective UML Modeling**: End-to-end alignment across Context, Use Case, Activity, Sequence, Class, and State Machine diagrams.
3. **Verifiable Non-Functional Requirements (NFRs)**: Concrete metric formulation based on Sommerville's NFR classification and the Goal-Question-Metric (GQM) framework.

1.2 Stakeholders & Actors

- **Customer**: Browses digital menus, customizes meal options, completes checkout and digital payments, tracks courier GPS movements in real time, and rates fulfilled orders.
- **Restaurant / Merchant**: Maintains digital menus, manages operational status, receives orders, and reports kitchen prep milestones.
- **Courier Partner**: Accepts dispatch invites via the mobile driver app, navigates to restaurants, picks up orders, and completes physical handovers to customers.

- **Platform Operations & Support Admin**: Monitors global dispatch health, handles order anomalies, arbitrates disputes, and authorizes refunds.
- **External Supporting Services**:
  - **Third-Party Payment Gateway**: Authorizes and settles credit card, Apple Pay, and digital wallet transactions (PCI-DSS compliant).
  - **Geospatial Map API**: Provides distance calculations, turn-by-turn routing, and Estimated Time of Arrival (ETA) predictions.
  - **Push Notification Service**: Delivers real-time APNs / FCM push notifications across client apps.

1.3 Key Terminology & Glossary

Term	Formal Definition	System Description
Order Entity	Transaction Contract	A binding transaction agreement created by a customer with a single restaurant, identified by a unique UUID.
Intelligent Dispatch	Dynamic Bipartite Matching	An algorithmic optimization engine matching available couriers to ready orders based on distance, rating, and batch capacity.
Kitchen Prep Time	Food Preparation Window	The estimated duration reported by the kitchen, governing the optimal courier arrival window to prevent cold food.
Idempotency	Idempotent Operation	A guarantee that duplicate API requests (e.g., repeated clicks on "Pay Now") execute exactly once without double charging.

---

2. General System Description

2.1 System Boundary & Context

QuickBite operates at the center of the food delivery ecosystem, coordinating transactions, communications, and real-time telemetry among customers, merchants, couriers, and cloud infrastructure:

External Actor / System	Interface Endpoint	Bidirectional Interaction with `QuickBite Core API`
1. Customer	Mobile App / Web SPA	Browses menus, submits orders, authorizes payments, and tracks live delivery status
2. Restaurant	Merchant Console	Receives order alerts, accepts orders, updates kitchen prep status, and marks ready for pickup
3. Courier Partner	Courier Driver App	Receives dispatch pings, confirms pickup/delivery, and streams GPS telemetry every 5 seconds
4. Geospatial Map API	Google Maps / Mapbox	Computes road distance matrices, dynamic ETA estimates, and turn-by-turn navigation routes

The standard System Context Diagram below formalizes the architectural boundary and external service contracts:

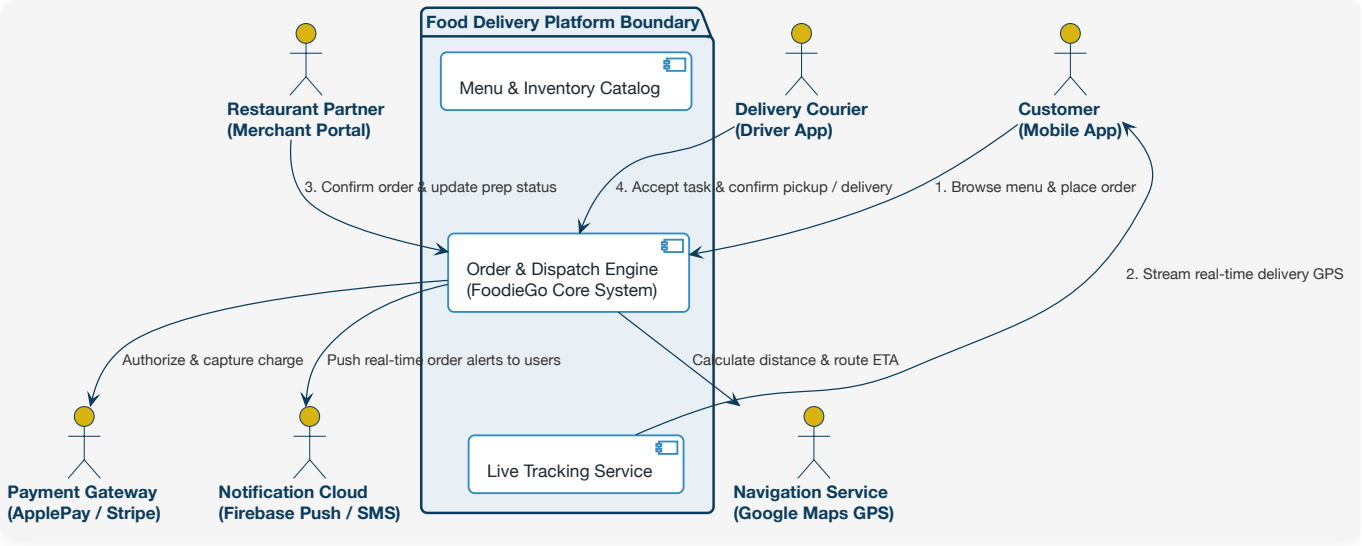


Figure 2.1 QuickBite Platform System Context Diagram

2.2 Assumptions & Dependencies

1. **Network Connectivity**: Couriers and customers are equipped with mobile devices featuring continuous cellular connectivity (4G/5G/Wi-Fi) and GPS location hardware.
2. **Traffic Surge**: Meal hours (11:30–13:30 and 17:30–19:30) experience 8× to 10× baseline transaction volumes.
3. **Regulatory Constraints**: Strict adherence to regional labor standards (maximum consecutive courier shift durations) and data privacy regulations (GDPR / consumer phone number masking).

---

3. Functional Requirements Specification

3.1 Overall Use Case Diagram

System functionality is structured around primary actors and the core fulfillment lifecycle:

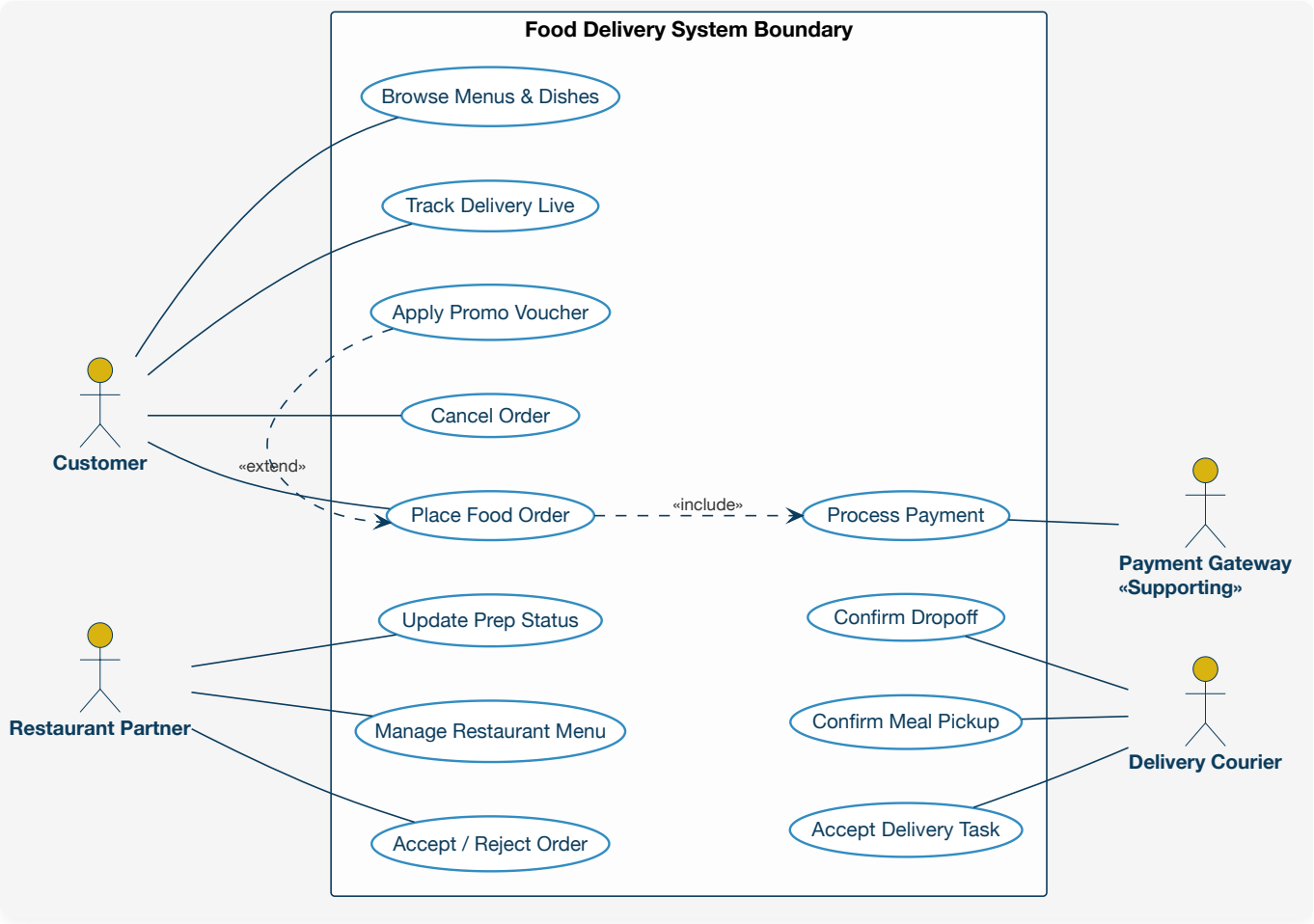


Figure 3.1 QuickBite Comprehensive Use Case Diagram

3.2 Use Case Relationships: Include vs. Extend Mechanics

Use case relationships express strict engineering semantics (aligned with Slide 04):

- **Include Relationship (`<<include>>`)**: An obligatory, non-optional component of the base use case. During "Checkout Order", the system **must** invoke "Process Payment".
- **Extend Relationship (`<<extend>>`)**: An optional behavior executed only when specific extension points and guard conditions are satisfied. When a customer enters a valid discount code, the system triggers "Apply Promo Code".

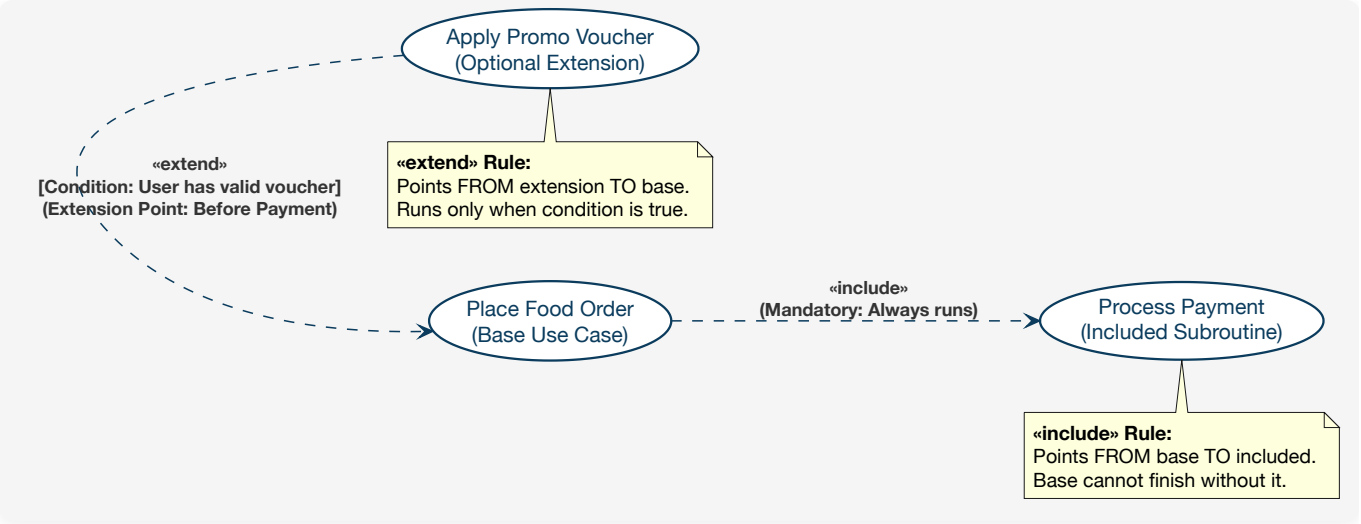


Figure 3.2 Detail of <> and <> Relationships in Order Checkout

3.3 Detailed Use Case Specifications

[UC-01] Browse Menu & Checkout Order

- **Primary Actor**: Customer
- **Preconditions**: Customer is authenticated and has set an active delivery address within operational service boundaries.
- **Main Success Scenario**:
  i. Customer launches the mobile app; nearby active restaurants are displayed sorted by rating and ETA.
  ii. Customer selects a restaurant and browses category menus.
  iii. Customer selects menu items, configures customizations (e.g., spice level, extra toppings), and adds them to the cart.
  iv. Customer navigates to checkout, reviewing order items, subtotal, delivery fee, and estimated arrival time.
  v. The system executes `<<extend>>`: Customer inputs a valid promo code `SE2026`; the system validates and recalculates the discounted total.
  vi. The system executes `<<include>>`: System initiates payment authorization via the third-party Payment Gateway.
  vii. Payment is authorized; system generates a unique order UUID and transitions state to `Placed`.
  viii. System sends real-time order alerts to the restaurant terminal via WebSocket and push notifications.
- **Alternative & Exception Flows**:
  - *4a. Out of stock*: An item becomes unavailable; system alerts the customer and removes it from the cart.
  - *6a. Payment declined*: Payment gateway rejects authorization; system preserves cart contents and prompts the customer for an alternative payment method.

- **Postconditions**: An immutable order record is persisted, funds are captured or pre-authorized, and the restaurant kitchen terminal chimes.

[UC-04] Accept Order & Kitchen Prep

- **Primary Actor**: Restaurant Kitchen Staff
- **Preconditions**: Order has passed payment authorization, with state set to `Paid`.
- **Main Success Scenario**:
  i. Restaurant terminal chimes and displays the new order card with item customizations and customer notes.
  ii. Kitchen staff verifies capacity, clicks "Accept Order", and inputs an estimated prep time (e.g., 20 minutes).
  iii. System transitions order state to `Accepted`, then to `Preparing`.
  iv. System triggers the dispatch service to search for optimal nearby couriers.
  v. Kitchen completes cooking and packages the food; staff clicks "Food Ready", transitioning state to `ReadyForPickup`.

[UC-05] Smart Dispatch & Order Fulfillment

- **Primary Actor**: Courier Partner, Dispatch Engine
- **Preconditions**: Order is in `Preparing` or `ReadyForPickup` state; active couriers are online nearby.
- **Main Success Scenario**:
  i. The dispatch algorithm selects the best candidate courier based on distance, traffic, and driver rating, issuing a 25-second countdown dispatch offer.
  ii. Courier clicks "Accept Order"; the order is assigned to the driver.
  iii. Courier arrives at the restaurant, verifies the 4-digit pickup code, receives the food, and clicks "Picked Up".
  iv. Order state transitions to `Delivering`; the system begins streaming real-time courier GPS coordinates to the customer app.
  v. Courier arrives at the destination, hands over the order, uploads proof of delivery, and clicks "Delivered".
  vi. Order state transitions to `Delivered`; system triggers digital receipt dispatch and prompts for dual-sided ratings.

3.4 Business Process Activity Diagrams

The food delivery lifecycle spans four distinct domains, modeled using UML Swimlane Activity Diagrams:

Phase 1: Customer Order Placement & Kitchen Preparation

Covers checkout validation, payment gateway authorization, and restaurant confirmation timeout safeguards:

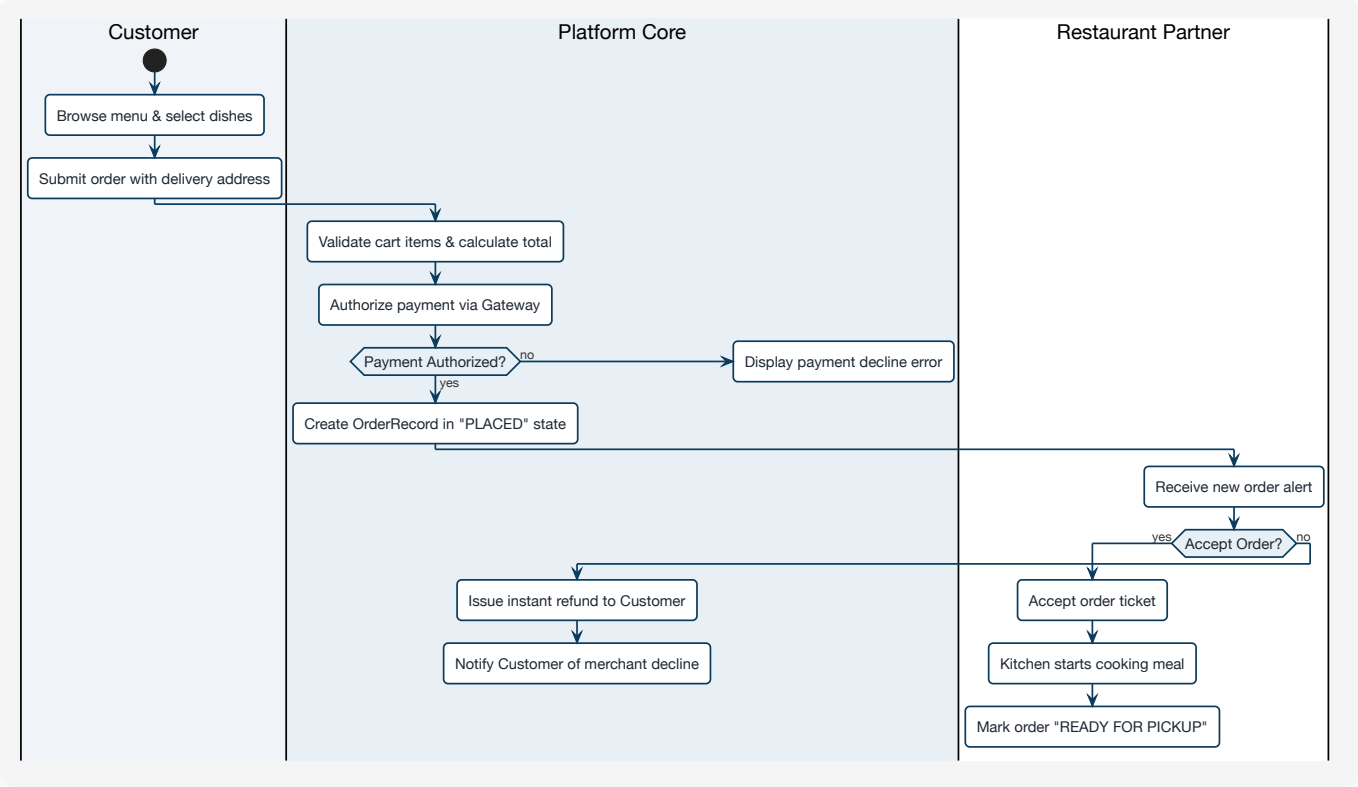


Figure 3.3 Phase 1: Customer Order, Payment Authorization & Kitchen Prep Activity Workflow

Phase 2: Courier Dispatch, Pickup & Delivery Handover

Illustrates concurrent kitchen preparation and courier dispatch, pickup verification, and final customer delivery:

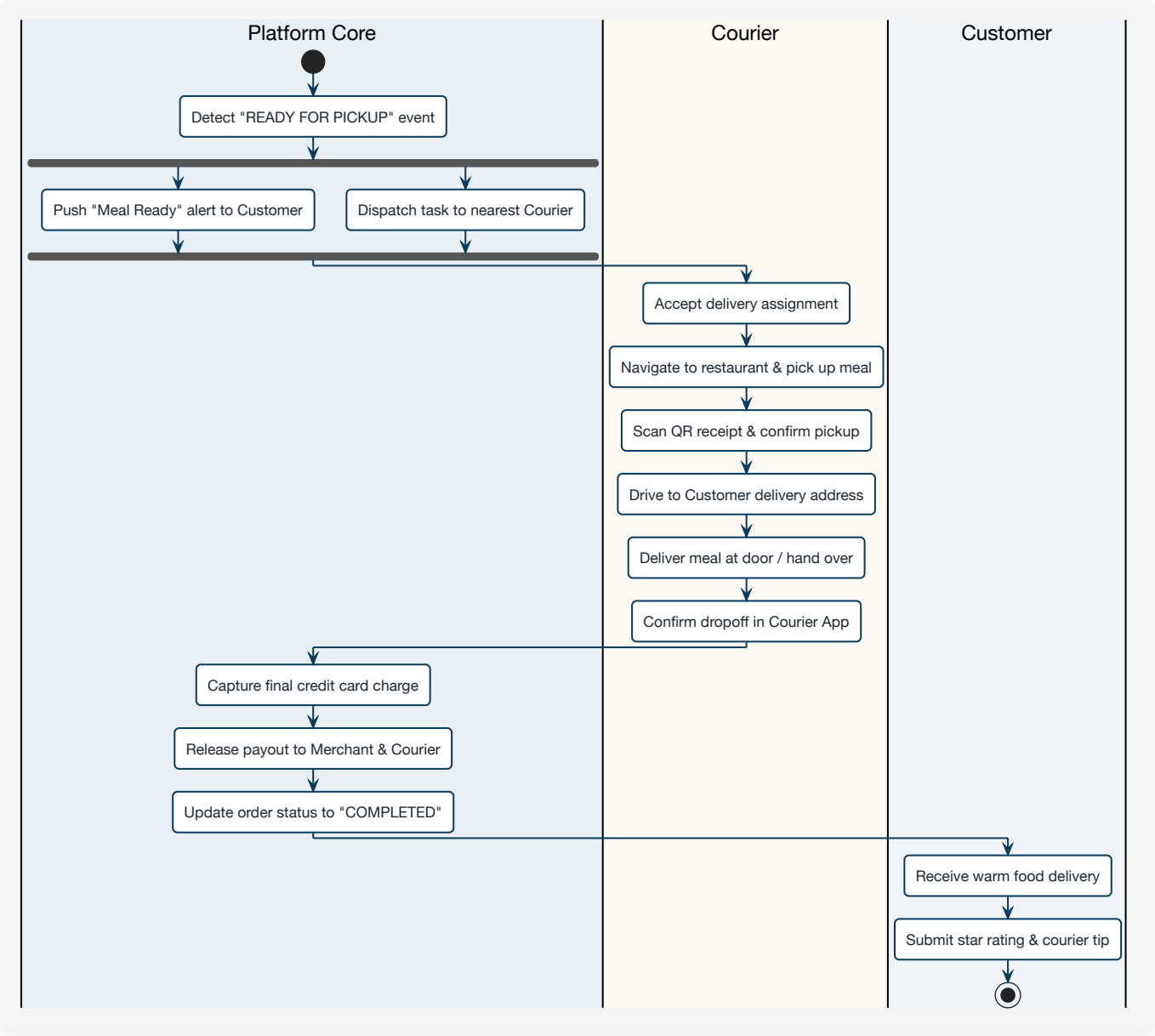


Figure 3.4 Phase 2: Courier Dispatch, Pickup & Delivery Handover Activity Workflow

---

4. Non-Functional Requirements (NFRs)

This section operationalizes the **Sommerville NFR Taxonomy** taught in **Slide 03 (Module 3.3)**, establishing quantifiable, testable engineering metrics:

NFR Taxonomy Family	Key Software Quality & Constraint Dimensions
1. Product Requirements	Performance & throughput, high availability & reliability, security & data masking, usability (UX)
2. Organizational Requirements	Development environment & framework standards, automated test coverage gates, CI/CD deployment standards
3. External Requirements	Municipal courier labor standards, electronic tax invoice compliance, national data privacy regulations

4.1 Product Requirements

4.1.1 Performance & Scalability

- **NFR-P-01 Read Latency**:
  - Under nominal load (3,000 read req/sec), restaurant and menu queries must maintain a 95th-percentile (P95) response time ≤ 200 ms and a 99th-percentile (P99) response time ≤ 400 ms.
- **NFR-P-02 Peak Transaction Throughput**:
  - The core checkout and payment processing pipeline must sustain at least 8,000 transactions per second (8,000 TPS); during peak festival promotions, Horizontal Pod Autoscalers (HPA) must scale capacity up to 15,000 TPS with error rates remaining ≤ 0.01%.
- **NFR-P-03 Real-Time Telemetry Update Frequency**:
  - Courier mobile apps must publish GPS coordinates every 3 seconds; customer-side map rendering latency must remain ≤ 2.0 seconds.

4.1.2 Reliability & Availability

- **NFR-A-01 Service Level Agreement (SLA)**:
  - Overall platform availability must achieve **99.95%** (unscheduled downtime must not exceed 4 hours and 23 minutes per calendar year).
- **NFR-A-02 Mean Time to Recovery (MTTR) & Fault Isolation**:
  - If any microservice instance crashes, Kubernetes liveness probes must detect the failure within 15 seconds and spin up a replacement pod; overall system MTTR must remain ≤ 5 minutes.
- **NFR-A-03 Order Idempotency Guarantee**:
  - Every checkout and charge request must mandate a client-generated `Idempotency-Key`. Under duplicate transmissions caused by network re-tries, the system guarantees 100% single execution, strictly preventing duplicate billing.

4.1.3 Security & Privacy Protection

- **NFR-S-01 Cryptographic Standards**:
  - All external and inter-service communications must enforce TLS 1.3 encryption.

    - User passwords must be hashed using Argon2id or bcrypt with strong work factors; sensitive financial tokens must be encrypted at rest using AES-256-GCM.
- **NFR-S-02 Virtual Phone Number Masking**:
    - Direct communications between couriers and customers must route through temporary virtual VoIP proxy numbers; neither party may view the other's real phone number or legal full name.
- **NFR-S-03 Financial Data Compliance (PCI-DSS)**:
    - Core platform databases must never store raw cardholder Primary Account Numbers (PAN) or CVV/CVC codes. Tokenization via certified payment partners is strictly mandatory.

4.1.4 Usability & Human-Centered UX (Aligned with Slide 06)

- **NFR-U-01 Checkout Efficiency**:
    - Returning customers re-ordering from a recent restaurant must complete checkout within ≤ 3 clicks, with a median completion duration ≤ 12 seconds.
- **NFR-U-02 Visibility of System Status (Nielsen #1)**:
    - Once placed, the customer view must prominently display a 5-stage visual progress tracker (Placed → Accepted → Preparing → In Transit → Delivered), reflecting state transitions within 1 second.
- **NFR-U-03 User Control & Freedom (Nielsen #3)**:
    - The customer checkout flow must provide a **10-second grace cancellation window**; until the restaurant clicks "Accept Order", customers retain the right to cancel with 100% automated refund.

4.2 Organizational Requirements

- **NFR-O-01 Automated Test Coverage Threshold**:
    - All core microservices (Order Calculation, Discount Engine, Dispatch Matching) must achieve ≥ 85% Statement Coverage and ≥ 75% Branch Coverage in automated unit test suites.
- **NFR-O-02 Continuous Integration & Deployment (CI/CD)**:
    - Every pull request must pass SonarQube static analysis, Trivy container security scans, and automated regression suites; total pipeline build-to-artifact duration must remain ≤ 8 minutes.

4.3 External & Regulatory Requirements

- **NFR-E-01 Labor Protection & Shift Limits**:
    - The dispatch service must enforce driver safety limits: after 8 cumulative active hours, warning reminders are issued; at 12 cumulative hours, the courier is forcibly set offline for at least 8 consecutive rest hours.
- **NFR-E-02 Electronic Invoice Integration**:

- Upon delivery completion, the platform must submit electronic invoice data to the tax authority API within 48 hours, supporting customer carrier barcodes and automatic receipts.

---

5. Verification & Acceptance Criteria

5.1 Gherkin BDD Acceptance Scenarios

BDD scenarios establish unambiguous acceptance contracts between engineering and business stakeholders:

```
Feature: Order Checkout and Idempotency Protection

  Scenario: Successful checkout with promo code deduction
    Given Customer "Alice" has "Truffle Mushroom Risotto" (1 portion, $250) in her cart
    And A valid promo code "SE2026" exists with rule: "$50 off on orders over $200"
    When Alice inputs promo code "SE2026" on the checkout page
    And Clicks "Confirm and Pay"
    Then The final payable amount must be calculated as $200
    And The payment gateway successfully charges $200
    And The order state transitions to "Placed"
    And The merchant terminal receives the new order alert within 2 seconds

  Scenario: Network jitter triggers rapid multiple clicks (Idempotency Guard)
    Given Customer "Bob" initiates checkout for $400 with Idempotency-Key "uuid-bob-1234"
    When Due to latency, Bob clicks "Confirm and Pay" 3 times within 1 second
    Then The payment gateway and order service process exactly 1 charge of $400
    And The database inserts exactly 1 order record
    And The subsequent 2 duplicate requests return the identical order confirmation with
status 200 OK
```

5.2 Goal-Question-Metric (GQM) Quality Matrix (Aligned with Slide 03)

Goal (G)	Question (Q)	Metric (M)	Acceptance Target
G1: Guarantee responsive checkout experience under peak surge	Q1.1 Do core APIs maintain low latency during meal peak hours?	Endpoint P95 / P99 latency (ms)	P95 ≤ 200 ms, P99 ≤ 400 ms
	Q1.2 Does peak concurrency result in dropped orders or server errors?	Transaction Error Rate (%)	Error rate ≤ 0.01%
G2: Maximize dispatch and delivery fulfillment efficiency	Q2.1 Is courier idle wait time minimized after arriving at restaurants?	Average courier wait duration for food	Average wait ≤ 3.5 minutes
	Q2.2 What is the latency of the dispatch matching computation?	Match engine batch execution duration	Computation time ≤ 1.5 seconds
G3: Ensure zero duplicate billing incidents across payment flows	Q3.1 How many customer tickets report duplicate charges due to re-tries?	Duplicate charge incident ticket volume	Exactly 0 duplicate charges per month

---

Authored by: Feng Chia University Software Engineering Teaching Team (Prof. Nien-Lin Hsueh)
Companion Document: QuickBite Software Design Document (SDD)